1) Цель

   Изучение принципов работы с системами виртуализации в облаке.

2) Задание

   a) Зарегистрировать аккаунт AWS или Yandex Cloud. При создании платежного аккаунта в Yandex Cloud воспользоваться стартовым грантом (или включить пробный период).

   b) Создать виртуальную машину:

      (i) AWS. В рамках уровня бесплатного использования AWS создать виртуальную машину на Amazon EC2 с использованием инстанса t2.micro под управлением ОС Linux. Использовать любой из дистрибутивов ОС Linux, предлагаемых Amazon в рамках программы бесплатного использования AWS.

      (ii) Yandex Cloud. В рамках пробного периода и предоставленного гранта Yandex Cloud создать виртуальную машину Yandex Compute Cloud под управлением ОС Linux. Использовать любой из дистрибутивов ОС Linux, предлагаемых Yandex Cloud в рамках пробного периода

   c) Запросить у преподавателя лично или по электронной почте доступ к администрированию репозитория лабораторной работы №5 на GitHub. В случае отправки запроса по электронной почте (m.polyak@guap.ru), указать в письме ссылку на свой репозиторий лабораторной работы №5, а также свой никнейм на гитхабе.

   d) Подготовить приложение к развертыванию в Kubernetes:

      (i) Создать Dockerfile для серверной части приложения.

      (ii) Создать в репозитории директорию k8s/ и разместить в ней следующие манифесты:

         (1) deployment.yaml — конфигурация развертывания с не менее чем 2 репликами сервера;

         (2) service.yaml — конфигурация сервиса для доступа к приложению;

         (3) configmap.yaml (опционально) — конфигурационные данные приложения.

      (iii) В манифесте deployment.yaml обязательно указать:

         (1) replicas: 2 или больше (минимум 2 реплики для демонстрации отказоустойчивости);

         (2) Ограничения ресурсов (requests и limits для CPU и памяти);

         (3) Переменные окружения, необходимые для работы приложения

   e) Установить и настроить Minikube на виртуальной машине в облаке:

(i) Установить Docker, kubectl и Minikube согласно инструкциям выше

(ii) Запустить Minikube-кластер командой minikube start –driver=docker

(iii) Проверить статус кластера: kubectl cluster-info и kubectl get nodes

(iv) Настроить Minikube для доступа извне (если используется NodePort)

f) Установить на виртуальной машине, созданной при выполнении п. 2, GitHub Runner и настроить CI/CD с автоматическим развертыванием в Minikube серверной части приложения ЛР №5. В случае, если по варианту ЛР №5 серверная часть приложения выполнялась в ОС Windows, использовать заглушку для серверной части приложения, разработанную в рамках ЛР №6.

(i) Установить на виртуальной машине GitHub Self-hosted Runner: https://docs.github
your-own-runners/managing-self-hostedrunners/adding-self-hosted-runners.

(ii) Создать или модифицировать файл .github/workflows/deploy.yml в репозитории (см. https://docs.github.com/ru/actions), прописав в нем последовательность шагов для сборки докер-контейнера с приложением, его деплоя с помощью minikube на сервер и верификации успешности деплоя.

(iii) При необходимости добавить секреты в репозиторий GitHub (Settings → Secrets and variables → Actions) для доступа к Docker Hub или другим ресурсам.

g) Настроить на виртуальной машине брандмауэр и открыть порт, на котором работает сервер. Настроить доступ к приложению через Minikube:

(i) Получить URL для доступа к сервису

(ii) Если используется NodePort, настроить правила брандмауэра для доступа к порту 30080 (или другому указанному в service.yaml).

(iii) Для Yandex Cloud/AWS настроить Security Group для разрешения входящего трафика на NodePort.

h) Продемонстрировать работу приложения:

(i) Продемонстрировать корректную работу клиентской части приложения с сервером, развернутым в Minikube на виртуальной машине. Это значит, что сервер должен быть запущен в облаке на виртуальной машине, а подключившийся к нему клиент должен работать на стационарном компьютере (или ноутбуке) в аудитории.

(ii) Показать работу механизма репликации

(iii) Продемонстрировать отказоустойчивость: удалить один под и показать, что Kubernetes автоматически создаст новый

i) Продемонстрировать автоматическое развертывание:

(i) Остановить текущее развертывание

(ii) Внести изменение в код или в конфигурацию (например, изменить количество реплик на 3)

(iii) Закоммитить изменения в репозиторий или запустить workflow вручную через интерфейс GitHub (Actions → Deploy to Minikube → Run workflow)

(iv) Дождаться завершения GitHub Actions workflow

(v) Проверить, что приложение автоматически развернулось с новой конфигурацией

(vi) Повторно продемонстрировать работу клиента с обновленным сервером

j) Мониторинг и логи:

(i) Продемонстрировать просмотр логов приложения

(ii) Показать базовые метрики (если установлен metrics-server)

k) После успешной защиты лабораторной работы остановить и удалить виртуальную машину, чтобы избежать автоматического списания денежных средств с банковской карты по окончании бесплатного периода использования AWS через 12 месяцев или окончании пробного периода Yandex Cloud через 60 дней.

3) Ход работы

a) Содержимое всех YAML-манифестов из директории k8s/

```
    apiVersion: apps/v1
kind: Deployment
metadata:
  name: server-deployment
  labels:
    app: server
spec:
  replicas: 3
  selector:
    matchLabels:
      app: server
  template:
    metadata:
      labels:
        app: server
    spec:
      containers:
      - name: server
        image: docker.io/library/server-app:latest
        imagePullPolicy: Never
```

```
        ports:
        - containerPort: 25001
        resources:
          requests:
            memory: "128Mi"
            cpu: "100m"
          limits:
            memory: "256Mi"
            cpu: "200m"


 apiVersion: v1
kind: Service
metadata:
  name: server-service
spec:
  type: NodePort
  selector:
    app: server
  ports:
  - protocol: TCP
    port: 8080
    targetPort: 25001
    nodePort: 30080
```

b) Содержимое файла .github/workflows/deploy.yml

```
    name: Deploy to minikube cloud
on:
  push:
    branches: [ main ]
  workflow_dispatch:
jobs:
  deploy:
    runs-on: lab7-hosted
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Build Docker image
        run: |
          docker build -t server-app:${{ github.sha }} -t server-app:latest .
      - name: Deploy to Minikube
```

```
      run: |
        # Загрузка образа в Minikube
        minikube image load server-app:latest
        # Применение манифестов из репозитория
        kubectl apply -f k8s/deployment.yaml
        kubectl apply -f k8s/service.yaml
        # Ожидание готовности подов
        kubectl rollout status deployment/server-deployment
        kubectl wait --for=condition=ready pod -l app=server --timeout=300s
    - name: Verify deployment
      run: |
        kubectl get deployments
        kubectl get pods
        kubectl get services

        # Проверка количества реплик
        REPLICAS=$(kubectl get deployment server-deployment -o
        jsonpath='{.spec.replicas}')
        echo "Deployed with $REPLICAS replicas"
        if [ "$REPLICAS" -lt "2" ]; then
          echo "Error: Less than 2 replicas configured!"
          exit 1
        fi
```

c) Скриншот успешного выполнения GitHub Actions workflow

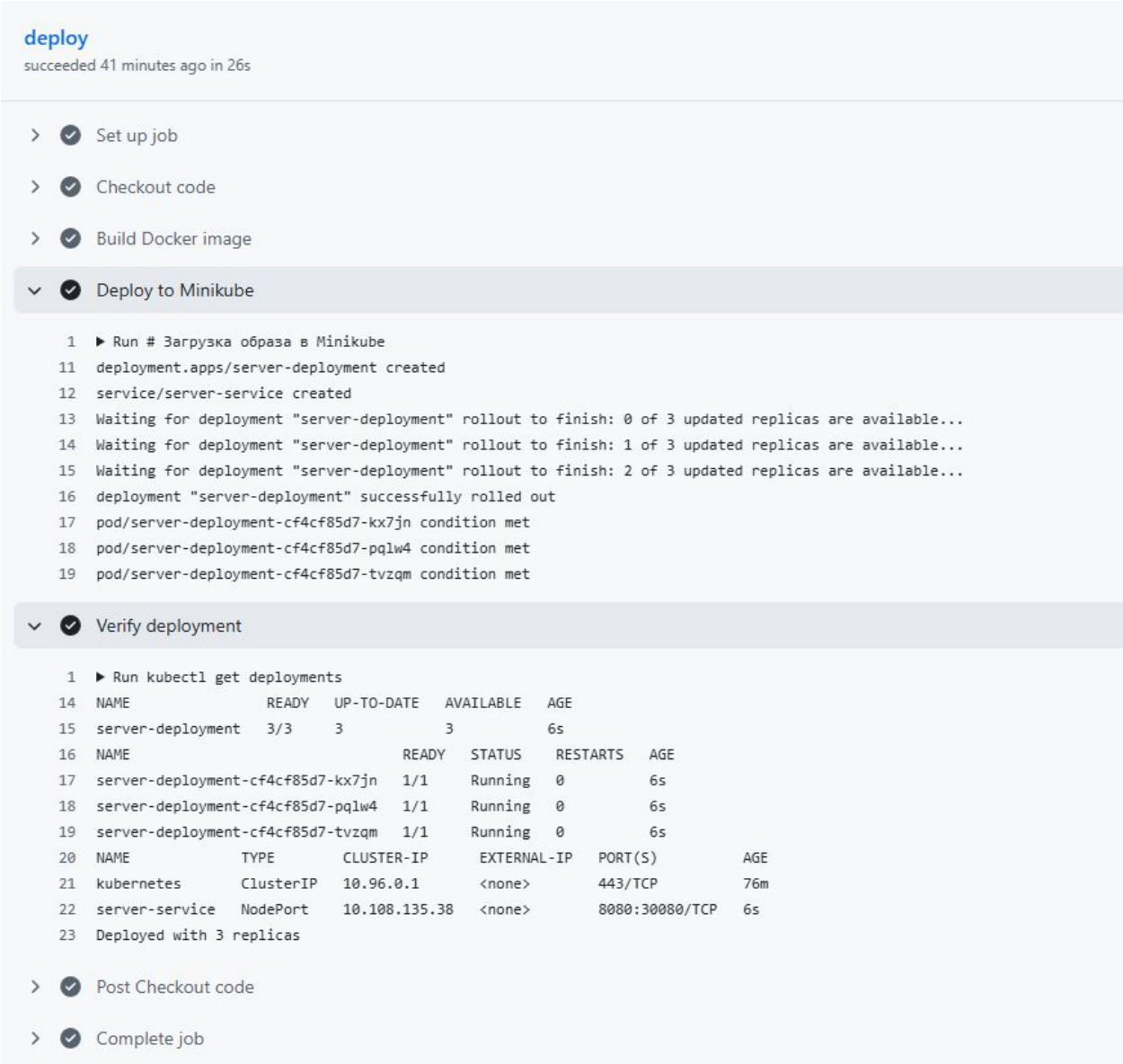


Рис. 1 – Рисунок выполненного workflow

d) Скриншот вывода команд

(i) kubectl get deployments

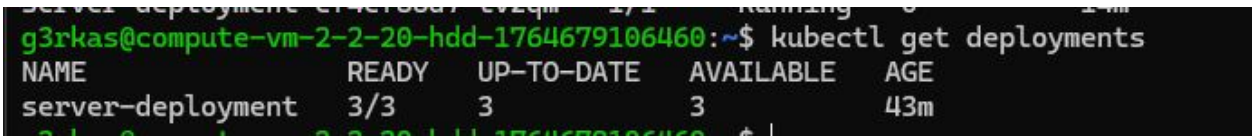


Рис. 2 – Рисунок результата команды kubectl get deployments

(ii) kubectl get pods

```
g3rkas@compute-vm-2-2-20-hdd-1764679106460:~$ kubectl get pods
NAME                                 READY   STATUS    RESTARTS   AGE
server-deployment-cf4cf85d7-9g7nq    1/1     Running   0          41m
server-deployment-cf4cf85d7-kx7jn    1/1     Running   0          44m
server-deployment-cf4cf85d7-tvzqm    1/1     Running   0          44m
```

Рис. 3 – Рисунок результата команды kubectl get pods

(iii) kubectl get services

```
g3rkas@compute-vm-2-2-20-hdd-1764679106460:~$ kubectl get services
NAME             TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
kubernetes       ClusterIP   10.96.0.1       <none>        443/TCP          121m
server-service   NodePort    10.108.135.38   <none>        8080:30080/TCP   44m
```

Рис. 4 – Рисунок результата команды kubectl get services

(iv) kubectl describe deployment server-deployment

```
g3rkas@compute-vm-2-2-20-hdd-1764679106460:~$ kubectl describe deployment server-deployment
Name:                   server-deployment
Namespace:              default
CreationTimestamp:      Tue, 02 Dec 2025 17:24:46 +0000
Labels:                 app=server
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=server
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=server
  Containers:
   server:
    Image:      docker.io/library/server-app:latest
    Port:       25001/TCP
    Host Port:  0/TCP
    Limits:
      cpu:     200m
      memory:  256Mi
    Requests:
      cpu:         100m
      memory:      128Mi
    Environment:   <none>
    Mounts:        <none>
  Volumes:         <none>
  Node-Selectors:  <none>
  Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Progressing    True    NewReplicaSetAvailable
  Available      True    MinimumReplicasAvailable
OldReplicaSets:  <none>
NewReplicaSet:   server-deployment-cf4cf85d7 (3/3 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  44m   deployment-controller  Scaled up replica set server-deployment-cf4cf85d7 f
rom 0 to 3
```

Рис. 5 – Рисунок результата команды kubectl describe deployment server-deployment

e) Скриншот работающего клиент-серверного приложения

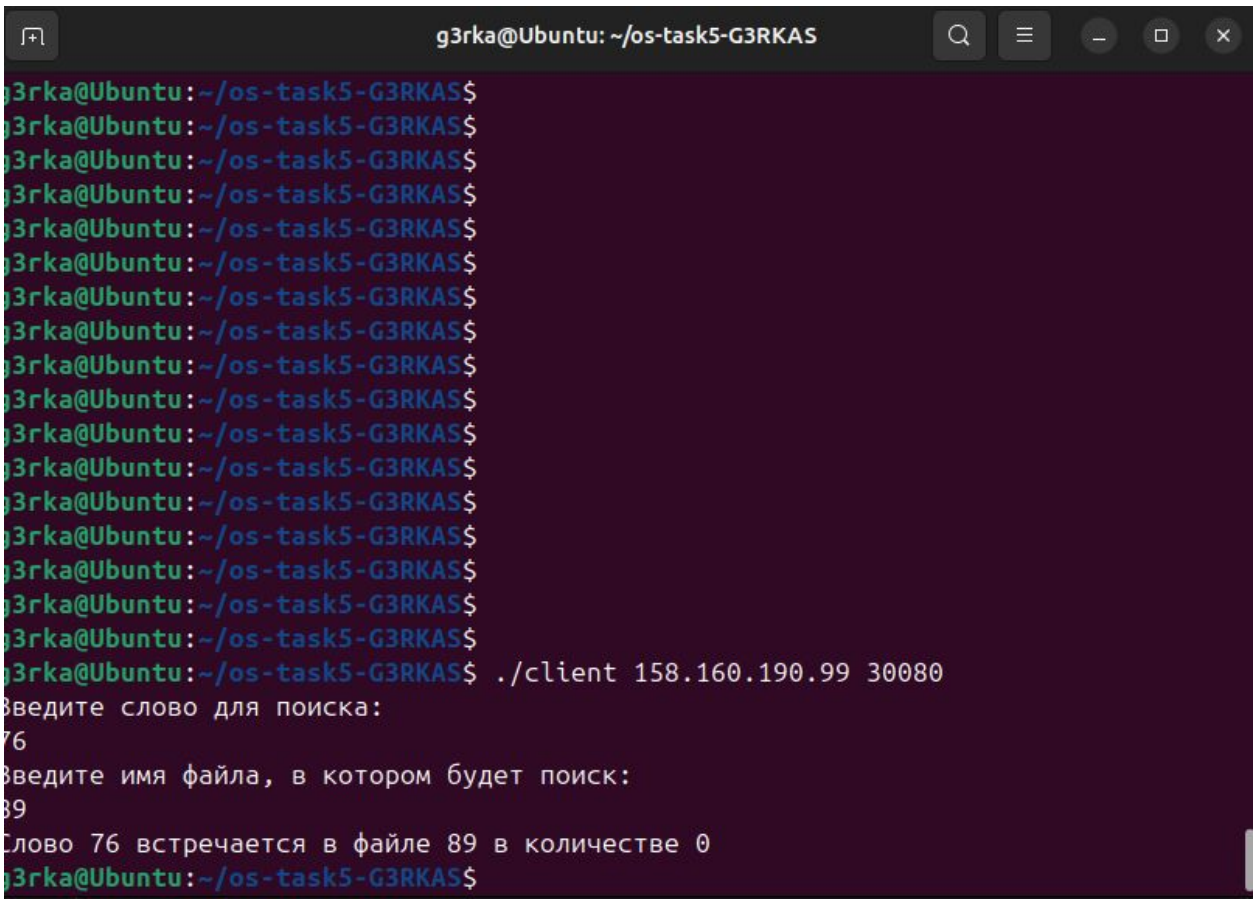


Рис. 6 – Рисунок работы приложения

f) Скриншот демонстрации отказоустойчивости (удаление пода и его автоматическое пересоздание)

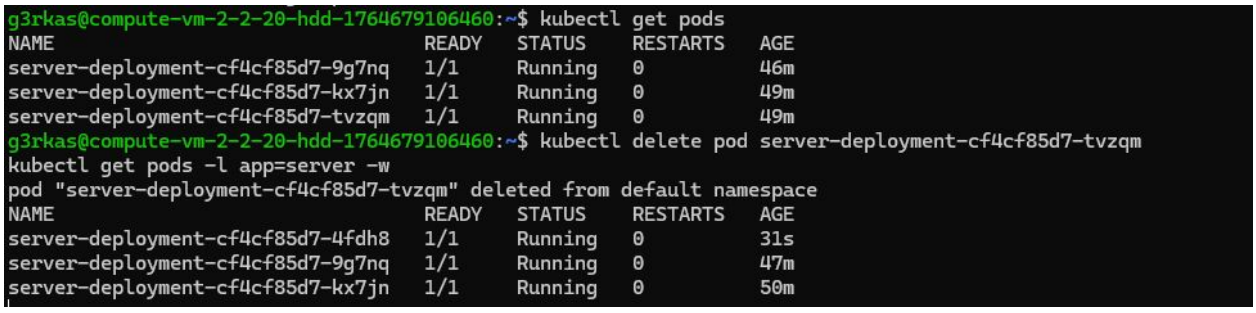


Рис. 7 – Рисунок работы отказоустойчивости

g) Скриншоты логов работы приложения

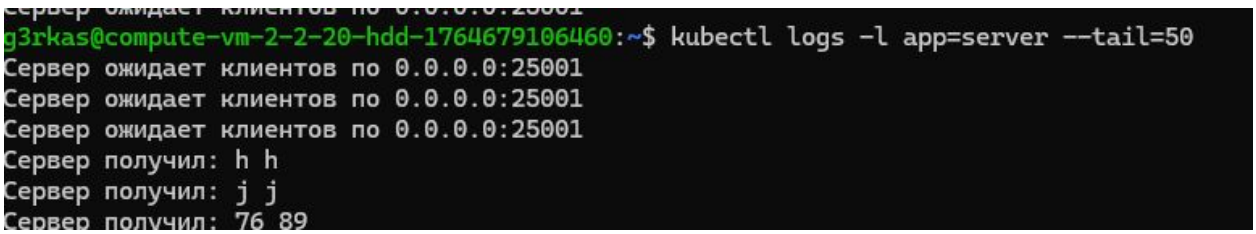


Рис. 8 – Рисунок работы логов

4) Исходный код

```
// Copyright 2025 Daniil Voroshilov
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <iostream>
#include <string>

using std::cout;
using std::endl;
using std::string;

int main() {
    int listen_sock = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_port = htons(25001);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);

    bind(listen_sock, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
    listen(listen_sock, 5);

    cout << "Сервер ожидает клиентов по 0.0.0.0:" << 25001 << endl;

    while (true) {
        int client_sock = accept(listen_sock, NULL, 0);  // отдельный дескриптор
        if (client_sock < 0) continue;

        char RecBuffer[512];
        int Bytes = recv(client_sock, RecBuffer, sizeof(RecBuffer)-1, 0);
        if (Bytes > 0) {
            RecBuffer[Bytes] = '\0';
            cout << "Сервер получил: " << RecBuffer << endl;

            string Reply = "0";
            send(client_sock, Reply.c_str(), Reply.size(), 0);
        }

        close(client_sock);
```

```
    }

    close(listen_sock);
    return 0;
}
```

5) Выводы

В процессе выполнения лабораторной работы были получены практические навыки развертывания приложений в облаке YandexCloud